BUILDING A FLASHCARD API FROM ZERO

FastAPI, Supabase & Multi-User Architecture — Complete Study Outline

I. WHY FASTAPI AND WHAT PROBLEM DOES IT SOLVE

A. The Core Problem: Static Scripts Cannot Serve External Clients

❐ 1. The Limitation of Local Python Scripts

- → **Principle**: A Python script that runs locally can process data, but it **cannot expose** that data to external applications (phones, browsers, other machines)
- A local script executes once, completes, and terminates — there is no persistent entry point for outside clients to connect to
- Even if the script reads a JSON file and processes flashcard data perfectly, there is no mechanism for an outside device to send it a request and receive a response
- **Context**: Imagine you have a script on your laptop that reads flashcard data from a JSON file and prints it to the terminal. It works perfectly when you sit at your laptop and run it. But the moment someone on a phone, or a web browser on another machine, wants to access that same data — the script has no "door" for them to knock on. It runs, it finishes, it is gone.

❐ 2. ✖ Why a Static File on a Web Server Is *Insufficient*

- → **Principle**: A static file hosted on a web server permits **retrieval only** — it cannot handle creation, modification, or deletion of data
- ✖ No ability to create new flashcards
- ✖ No ability to edit existing flashcards
- ✖ No ability to delete flashcards
- The file is inert — it does not differentiate between types of requests; it simply serves the same content to everyone who asks

❐ 3. ✔ The Solution: A *Living* API

- → **Principle**: An **API (Application Programming Interface)** is a persistent program that sits on a server, listens for incoming requests, processes them, communicates with a database, and returns structured responses

- **Context**: "API" stands for Application Programming Interface. Think of an API as a receptionist at a front desk who is always present. Visitors (requests) arrive, they state what they need (retrieve data, create something, update something, delete something), the receptionist goes to the filing cabinets (database), performs the requested action, and hands back the result. Without the receptionist, the filing cabinets exist but nobody from outside can interact with them. The API is what makes data **accessible and modifiable** by external applications.

B. How an API Works in Practice: Endpoints, HTTP Methods & CRUD

❒ 1. Endpoints = Specific URLs That Accept Requests

- **→ Principle**: An API exposes **endpoints** — specific URLs that external clients can send requests to, each representing a different resource or action
- An endpoint is simply a URL like `localhost:8000/cardfolders`
- Different endpoints correspond to different resources (users, folders, card sets, cards)

❒ 2. HTTP Methods = The Verb That Tells the API What To Do

- **→ Principle**: The **HTTP method** attached to the request determines which operation the API performs on the resource identified by the endpoint
- **Context**: HTTP (HyperText Transfer Protocol) is the communication protocol used by web browsers and servers to talk to each other. Every time you visit a website, your browser sends an HTTP request. Each request includes a "method" — a verb that tells the server what kind of action the client wants. In everyday browsing, your browser almost always sends GET requests (to retrieve web pages). But APIs use the full range of methods to support different operations.

HTTP METHOD	CRUD OPERATION	WHAT IT DOES
GET	**R**ead	Retrieve existing data without modifying anything
POST	**C**reate	Send new data to the server to be stored
PUT	**U**pdate	Modify an existing resource with new data
DELETE	**D**elete	Remove an existing resource

- **→ Principle**: These four operations — **Create, Read, Update, Delete (CRUD)** — are the fundamental building blocks of virtually every data-driven application
- The acronym **CRUD** maps directly to the four HTTP methods: POST → Create, GET → Read, PUT → Update, DELETE → Delete

C. Why FastAPI Over Other Python Frameworks

❒ 1. The Landscape: Flask, Django REST Framework, FastAPI

- **Context**: Python has several popular frameworks for building web APIs. **Flask** is the oldest and most well-known — it is a "micro-framework," meaning it gives you the bare minimum and you add everything else yourself. **Django REST Framework** is built on top of Django, a full-featured web framework — it is powerful but heavyweight, with a steep learning curve. **FastAPI** is the newest of the three, released in 2018, and was designed from the ground up to leverage modern Python features.

❒ 2. ✔ Reason 1: Automatic Data Validation via Pydantic

- ➔ **Principle**: FastAPI uses a library called **Pydantic** to automatically validate incoming request data against predefined models — rejecting malformed requests before your code even runs
- **Context**: "Pydantic" is a Python library (its name is a play on "pedantic") that lets you define data models as Python classes with type annotations. You say "this field must be a string, this field must be an integer, this field is optional." When data arrives that does not match the model, Pydantic immediately rejects it with a detailed error message. You do not write a single line of checking code yourself.
- In Flask, you would write all validation logic by hand — checking each field, its type, whether it is present, whether it is the right format
- In FastAPI, you define a model once and validation happens automatically
- ✖ Bad data never reaches your database — it is caught at the API boundary with an automatic **422 Unprocessable Entity** response

❒ 3. ✔ Reason 2: Automatic Interactive Documentation (Swagger UI)

- ➔ **Principle**: The moment a FastAPI server starts, it auto-generates a fully interactive documentation page at `/docs` — no configuration, no extra code required

- **Context**: **Swagger UI** is an open-source tool that renders API documentation as an interactive web page. You can see every endpoint, the data each one expects, and you can test them directly in your browser by clicking "Try it out." Traditionally, developers use separate tools like **Postman** (a desktop application for sending API requests) or **curl** (a command-line tool for making HTTP requests). FastAPI eliminates the need for these during development by building the documentation automatically from your Python code.
- FastAPI reads your Python type hints and Pydantic models and uses them to generate this page
- Available at `localhost:8000/docs` as soon as the server runs

❒ 4. ✔ Reason 3: Modern Python Type Hints & Async Support

- **→ Principle**: FastAPI uses Python's native type hint system for **dual purpose** — the same type annotation drives both validation and documentation simultaneously
- **Context**: **Type hints** were introduced in Python 3.5 (PEP 484). They are optional annotations that indicate what type a variable, parameter, or return value should be — for example, `name: str` means "name should be a string." Python does not enforce them at runtime by default, but libraries like Pydantic and frameworks like FastAPI read them and act on them. This means a single line like `fronttext1: str` simultaneously tells FastAPI to validate that the incoming value is a string AND to document it as a string in the Swagger page.
- FastAPI also supports **async** (asynchronous) code — meaning it can handle many requests concurrently without blocking
- **Context**: "Async" refers to asynchronous programming, a paradigm where the program can start a task (like waiting for a database response) and move on to handle other requests while waiting, rather than sitting idle. This tutorial uses synchronous (sequential) code for simplicity, but the async capability exists for high-performance applications.

D. Why Supabase Over a Local SQLite File

❐ 1. What Supabase Is

- **Context**: **Supabase** is an open-source platform that provides a hosted **PostgreSQL** database with a visual web dashboard, authentication services, and client libraries for multiple languages (Python, JavaScript, etc.). **PostgreSQL** (often called "Postgres") is one of the most powerful and widely-used relational databases in the world — used by companies like Instagram, Spotify, and Netflix. Supabase essentially wraps PostgreSQL in a user-friendly interface so you can create tables through a web dashboard instead of writing raw SQL commands. **SQLite**, by contrast, is a lightweight database that stores everything in a single file on your local machine — perfect for small personal projects but limited because only your machine can access it.

❐ 2. Three Advantages of Supabase Over SQLite

- ✓ **Remote accessibility**: The database is accessible from anywhere with an internet connection — not just the machine it is running on
- ✓ **Visual dashboard**: You can inspect, query, and modify your data through a graphical web interface in real time
- ✓ **Clean Python client library**: Database queries are written using a chained method syntax (e.g., `supabase.table("cards").select("*").execute()`) that is readable and consistent
- ➜ **Principle**: When you later deploy your API to a production server, the Supabase connection works identically — no migration or reconfiguration needed

E. The Multi-User Problem: Four Features That Transform a Toy Into a Platform

❐ 1. ✓ Feature 1 — *Ownership*: Who Owns What

- ➜ **Principle**: Every writable resource (folder, card set) must have a designated **owner**, and every write operation (create, update, delete) must verify that the requesting user **is** the owner before proceeding
- If User A creates a card set called "Spanish Verbs," User B must not be able to edit or delete it
- Ownership is enforced at the API level — the check happens before any database modification occurs

❐ 2. ✓ Feature 2 — *Visibility*: Public vs. Private

- ➜ **Principle**: Each card set carries a boolean flag (`is_public`) that controls whether it is visible to users other than the owner — **private by default**
- ✖ Private sets: invisible to everyone except the owner
- ✓ Public sets: browsable and viewable by all users, including anonymous ones

❐ 3. ✓ Feature 3 — *Audit Trails*: When and Who

- ➜ **Principle**: Every card modification must be stamped with the **timestamp** of the change (`updatedat`) *and the **identity** of the user who made it* (*`updated`* `by`), creating a permanent record of edits
- If a card's text gets corrupted or an accidental edit occurs, the audit trail identifies when it happened and who did it

❐ 4. ✓ Feature 4 — *Copying*: Forking Public Content

- ➜ **Principle**: If a user discovers a public card set they want to personalise, the API supports a **one-click deep copy** that duplicates the entire card set and all its cards into the requesting user's account as an independent copy
- The copy belongs entirely to the new user — they can edit, delete, add, or

make it private
- The original is completely unaffected

❐ 5. Summary of Section I

- ✖ Static files cannot handle CRUD operations from external clients → an API is required
- ✔ FastAPI chosen for: automatic Pydantic validation, automatic Swagger documentation, modern type hints
- ✔ Supabase chosen for: remote PostgreSQL hosting, visual dashboard, clean Python client
- ✔ Multi-user features required: ownership, visibility, audit trails, copying
- The tutorial comprises eight sections covering setup through full deployment of all 22 endpoints

---

II. ENVIRONMENT SETUP ON macOS

A. Overview: Four Setup Steps

- → **Principle**: The environment setup follows a strict sequence — create isolation first (virtual environment), install dependencies second, configure secrets third, verify fourth
- Step 1: Create a project folder and activate the Python virtual environment
- Step 2: Install FastAPI, Uvicorn, Supabase client, and python-dotenv
- Step 3: Create a `.env` file for Supabase credentials and a `.gitignore` for protection
- Step 4: Verify everything works with a quick test server

B. Step 1: Project Folder and Virtual Environment

❒ 1. Terminal Commands (Executed in Sequence)

- `mkdir flashcard-api` — creates the project folder
- `cd flashcard-api` — navigates into it
- `workon vintel` — activates the virtual environment
- After activation, the terminal prompt changes to show `(vintel)` at the beginning — this confirms you are inside the virtual environment

❒ 2. What a Virtual Environment Is and Why It Matters

- **Context**: A **virtual environment** is an isolated Python installation managed by **virtualenvwrapper**. Think of it as a sealed room: any libraries you install while inside this room go onto shelves in that room only — they do not touch the shelves in any other room (other projects) or in the main hallway (your system Python). This prevents "dependency conflicts," where Project A needs version 2 of a library but Project B needs version 3, and installing one breaks the other. The `workon` command is provided by virtualenvwrapper and activates a named environment stored in a central location (typically `~/.virtualenvs/`), keeping your project folder clean.
- **→ Principle**: The activation command (`workon vintel`) **does not persist** between terminal sessions — every time you open a new terminal window to work on this project, you must run it again

C. Step 2: Installing the Four Packages

❒ 1. The Install Command

- With the virtual environment active (you see `(vintel)` in your prompt), run:

```
pip install fastapi uvicorn supabase python-dotenv
```

- This single command installs all four packages and their dependencies

❒ 2. What Each Package Does

❒ a. `fastapi` — The Web Framework

- Provides the decorators (`@app.get` , `@app.post` , etc.) that turn ordinary Python functions into API endpoints
- Handles request routing, parameter parsing, and response serialisation
- ✖ Cannot listen for HTTP requests on its own — it is a set of rules and decorators, not a server

❒ b. `uvicorn` — The ASGI Server (The Engine)

- **Context**: **ASGI** stands for **Asynchronous Server Gateway Interface**. It is a specification (a standard) that defines how a Python web application (like FastAPI) communicates with a web server (like Uvicorn). Think of it as a contract: FastAPI speaks ASGI, Uvicorn speaks ASGI, so they can work together. The predecessor to ASGI is **WSGI** (Web Server Gateway Interface), which does not support asynchronous operations. Uvicorn is an ASGI server, meaning it understands both synchronous and asynchronous Python code.
- ➜ **Principle**: Uvicorn is the piece that **listens on a network port** (port 8000 by default), receives incoming HTTP requests, hands them to FastAPI for processing, and sends the responses back to the client
- FastAPI defines *what* to do with requests; Uvicorn is *how* those requests arrive and depart

❐ c. `supabase` — The Database Client Library

- The official Python client for communicating with your hosted Supabase PostgreSQL database
- Provides a chained method API: `supabase.table("cards").select("*").execute()`

❐ d. `python-dotenv` — The Environment Variable Loader

- **Context**: **Environment variables** are key-value pairs that exist in your operating system's session, accessible by any program running in that session. They are commonly used to store configuration values (like database URLs and API keys) outside of source code. The `.env` file is a simple text file where each line is `KEY=VALUE`. The `python-dotenv` library reads this file and loads its contents into the Python process's environment variables, making them accessible via `os.environ["KEY_NAME"]`.
- **→ Principle**: Credentials must **never** appear in source code — `python-dotenv` allows you to store them in a separate `.env` file that is excluded from version control

---

D. Step 3: Configuring Credentials and Protection

❒ 1. Creating the `.env` File

- Create a file named `.env` (note the leading dot — this makes it a hidden file on macOS) in the project root
- Contents — two lines:
- `SUPABASE_URL=<your project URL>`
- `SUPABASE_KEY=<your anon public key>`

❒ 2. Where to Find Your Supabase Credentials

- Go to `supabase.com` → open your project → click **Project Settings** in the left sidebar → click **API**
- Two values are displayed: the **Project URL** and the **anon public key**
- Copy both into the `.env` file

❒ 3. Creating the `.gitignore` File

- **Context**: **Git** is a version control system that tracks changes to your code files. **GitHub** is a cloud platform where you can upload (push) your Git repositories so others can see and collaborate on your code. A `.gitignore` file tells Git which files and folders to **exclude** from tracking — meaning they will never be uploaded to GitHub, even accidentally.
- **➜ Principle**: Three items must **always** be excluded from version control in this project:
- `venv/` — any local virtual environment folder, if present (large, machine-specific, and reproducible via `pip install`)
- `.env` — your secret credentials (exposing these publicly would let anyone access your database)
- `pycache/` — Python's compiled bytecode cache (automatically generated, not source code)

E. Step 4: Verification

❒ 1. Creating a Minimal Test Server

- Create a file `test_setup.py` in the project root
- Inside: import FastAPI, create an `app` instance, add one GET endpoint at `/` that returns `{"message": "Flashcard API is alive"}`

❒ 2. Running It with Uvicorn

- Command: `uvicorn test_setup:app --reload`
- **Context**: The format of this command is `uvicorn <filenamewithoutextension>:<variablename>`. *So* `test` `setup:app` tells Uvicorn to look inside `test_setup.py` for a variable called `app`. The `--reload` flag enables **hot reloading** — Uvicorn watches your files for changes and automatically restarts the server whenever you save, so you do not need to manually stop and restart during development.
- Open browser → `localhost:8000` → you should see the JSON message
- Open browser → `localhost:8000/docs` → you should see the interactive Swagger UI with one endpoint listed
- Press `Ctrl+C` in the terminal to stop the server
- Delete `test_setup.py` — it was only for verification

❒ 3. Summary of Section II

- Project folder created with isolated virtual environment
- Four packages installed: `fastapi` (framework), `uvicorn` (server), `supabase` (database client), `python-dotenv` (credential loader)
- `.env` file holds Supabase credentials; `.gitignore` protects them from being committed to version control
- Verification confirms the server runs and Swagger documentation auto-generates
- Current project contents: `.env`, `.gitignore`

⌘ Project Initialisation — Terminal Commands

Commands: Create project directory and install dependencies

Description: These terminal commands set up the project from scratch. First, a project directory is created and entered. The virtual environment is activated using virtualenvwrapper's `workon` command. Then the four required Python packages are installed: `fastapi` (the web framework), `uvicorn` (the ASGI server), `supabase` (the Python client for the Supabase database), and `python-dotenv` (loads credentials from `.env` files into environment variables).

Create the project and activate the virtual environment:

```
mkdir flashcard-api
cd flashcard-api
workon vintel
```

Install the required packages:

```
pip install fastapi uvicorn supabase python-dotenv
```

Notice: The command `workon vintel` uses virtualenvwrapper (not the standard `python -m venv`) to activate a pre-existing virtual environment named `vintel`. If you use a standard venv instead, replace this with `python -m venv venv && source venv/bin/activate`. The important thing is that packages install into an isolated environment, not your global Python installation.

⌘ Environment Configuration — Credentials and Git Safety

Files: `.env` **and** `.gitignore`

Description: These two configuration files work together to manage sensitive credentials securely. The `.env` file stores your Supabase project URL and API key — the two values the Python client needs to connect to your database. The `.gitignore` file ensures that `.env` is never committed to version control, keeping your credentials out of your repository. It also excludes Python's `pycache/` bytecode directory.

File: `.env`

```
SUPABASE_URL=https://your-project-id.supabase.co
SUPABASE_KEY=your-anon-key-here
```

File: `.gitignore`

```
.env
__pycache__/
```

Notice: Replace the placeholder values in `.env` with your actual Supabase credentials from the Supabase dashboard (Settings → API). The `SUPABASEKEY` *here refers to the* `anon` *(public) key, not the* `service` `role` key. Never commit the `.env` file to Git — if it is ever exposed, regenerate your keys immediately from the Supabase dashboard.

⌘ Setup Verification — testsetup.py

File: `test_setup.py` **(temporary — delete after testing)**

Description: This is a minimal FastAPI application used solely to verify that your environment is correctly configured before building the real API. It creates a single endpoint at the root URL (`/`) that returns a simple JSON message. Running it with `uvicorn` confirms that FastAPI and Uvicorn are properly installed and that the development server starts without errors. Once verified, this file should be deleted — the real entry point will be `main.py`.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "Flashcard API is alive"}
```

Run the verification server:

```
uvicorn test_setup:app --reload
```

Notice: The `--reload` flag tells Uvicorn to watch for file changes and automatically restart the server — essential during development but should never be used in production. After running, visit `http://127.0.0.1:8000` to see the JSON response, and `http://127.0.0.1:8000/docs` to verify that Swagger documentation auto-generates. Delete `test_setup.py` once both checks pass.

III. DATABASE DESIGN IN SUPABASE

A. The Data Hierarchy: Four Tables, Strict Parent-Child Relationships

❒ 1. Overview of the Four Tables

- → **Principle**: The database consists of four tables created in a specific order — each subsequent table references the one before it, so **creation order matters** (you cannot reference a table that does not yet exist)

TABLE	PURPOSE	DEPENDS ON
`users`	Identity — who uses the system	None (created first)
`cardfolders`	Grouping — containers for card sets	`users`
`cardsets`	Collections — sets of flash-cards with ownership & visibility	`users` , `cardfolders`
`cards`	Content — individual flash-cards with audit trails	`cardsets` , `users` (for audit)

❒ 2. The Hierarchy in Plain Language

- **Users** sit at the foundation — they have a unique ID, username, and email
- **Card folders** group related collections — e.g., "Spanish Vocabulary" or "Medical Terms." Each folder is owned by exactly one user
- **Card sets** live inside folders — e.g., the "Spanish Vocabulary" folder might contain card sets like "Food Words," "Colors," and "Verbs." Each card set has an owner and a public/private visibility flag
- **Cards** live inside card sets — individual flashcards with front/back content. They carry audit trail fields showing when and by whom they were last modified

B. Why Not One Giant Table? The Case for Normalisation

❒ 1. ✖ The *Flat Table* Anti-Pattern: Four Problems

- → **Principle**: Storing all data in a single table (one row per card, with folder name, set name, and user details repeated in every row) creates four categories of serious problems known as **data anomalies**

❒ a. ✖ Problem 1 — Data Duplication

- If a folder called "Spanish Vocabulary" contains 500 cards across 10 sets, the string "Spanish Vocabulary" is stored 500 times
- Renaming the folder requires updating 500 rows instead of one
- Wasted storage and wasted processing

❒ b. ✖ Problem 2 — Deletion Anomalies

- If you delete all cards in a set, the set itself vanishes because there is no row left to represent it
- The user may have wanted to keep the empty set as a placeholder — but the set's existence was entirely dependent on having at least one card row

❒ c. ✖ Problem 3 — Inconsistency

- One row says "Spanish Vocabulary," another says "Spanish Vocabulay" (typo)
- There is no single authoritative source for the folder name — each row is a separate copy that can diverge

❒ d. ✖ Problem 4 — User Data Duplication

- Without a separate users table, every card row repeats the user's name and email
- If a user changes their email, every card they ever created or edited requires updating

❐ 2. ✔ The Solution: *Normalisation*

- ➜ **Principle**: **Normalisation** is the practice of splitting data into separate tables so that each fact is stored **exactly once** — everything else stores a reference (a foreign key) pointing to that single source of truth
- **Context**: Normalisation is a core concept in relational database design, originally formalised by Edgar F. Codd in 1970. The basic idea is to eliminate redundancy. Instead of repeating "Spanish Vocabulary" 500 times across card rows, you store it once in a `cardfolders` table and give it a unique ID. Every card row then stores only that ID — a lightweight pointer — instead of the full string. Need to rename the folder? Change one row. Every card that references it automatically "sees" the new name.

C. Foreign Keys: The Glue Between Tables

❐ 1. What a Foreign Key Is

- **→ Principle**: A **foreign key** is a column in one table that stores a value matching the **primary key** of another table — establishing a parent-child relationship between them
- **Context**: A **primary key** is a column (or set of columns) that uniquely identifies every row in a table — no two rows can have the same primary key value. In our design, every table's primary key is the `id` column (a UUID). A **foreign key** is a column in a *different* table that references a primary key. For example, the `cardfolders` table has a `user_id` column — each value in that column matches an `id` in the `users` table, establishing "this folder belongs to this user."
- The database **enforces** foreign key constraints — you cannot insert a value in a foreign key column that does not correspond to an existing row in the referenced table

❐ 2. ON DELETE Behaviours: CASCADE vs. SET NULL

- **→ Principle (CASCADE)**: Use `ON DELETE CASCADE` when the child entity **cannot meaningfully exist** without its parent — deleting the parent automatically deletes all children

- **→ Principle (SET NULL)**: Use `ON DELETE SET NULL` when the foreign key is merely a **reference for informational purposes** — deleting the referenced entity clears the reference but preserves the child

SCENARIO	BEHAVIOUR	RATIONALE
User deleted → their folders	CASCADE	Folders cannot exist without an owner
Folder deleted → its card sets	CASCADE	Card sets cannot exist without a parent folder
Card set deleted → its cards	CASCADE	Cards cannot exist without a parent set
User deleted → `updated_by` on cards they edited	SET NULL	The card is still valid; only the editor reference becomes null ("the user who last edited this card no longer exists")

D. Multi-User Columns: Three Categories

❐ 1. ✓ Ownership Columns

- → **Principle**: Both `cardfolders` and `cardsets` carry a `user_id` column pointing to the `users` table — every write operation checks this column against the requesting user, returning **403 Forbidden** if they do not match
- Cards do not have their own `user_id` — ownership is inherited from the parent card set

❐ 2. ✓ Visibility Columns

- → **Principle**: The `cardsets` table carries an `is_public` boolean that defaults to `false` — the owner must **explicitly** flip it to `true` to share
- Visibility lives at the card set level, **not** the folder level
- **Context**: If visibility were on folders, you could have contradictions — for instance, a public card set inside a private folder. Should it be visible or not? Keeping visibility exclusively on card sets avoids all such ambiguity.

❐ 3. ✓ Audit Columns

- → **Principle**: The `cards` table carries `updatedat` *(timestamptz, initially null) and* `updated` `by` (uuid, initially null, FK → users with SET NULL) — every modification stamps both fields automatically
- A null `updated_at` clearly communicates: "this card has never been edited since creation"

E. Creating the Four Tables in Supabase (Step-by-Step)

❒ 1. Navigating the Supabase Dashboard

- **Context**: When you log into `supabase.com` and open your project, the left sidebar shows several options including Authentication, Database, Storage, and others. The one you need is **Table Editor** — this provides a graphical interface for creating tables and columns without writing SQL. Click **New Table** to begin. **Important**: Uncheck "Enable Row Level Security" for now — security will be handled at the API level for this tutorial.

❒ 2. Table 1: `users` (Created First — No Dependencies)

COLUMN	TYPE	CONSTRAINTS	DEFAULT	NOTES
`id`	uuid	PRIMARY KEY, NOT NULL	`gen_random_uuid()`	Auto-generates a unique ID for every row
`username`	text	NOT NULL, UNIQUE	—	UNIQUE prevents duplicate usernames at the database level
`email`	text	NOT NULL, UNIQUE	—	UNIQUE prevents duplicate emails at the database level
`created_at`	timestamptz	—	`now()`	Automatically stamps the creation time

- **Context**: **UUID** stands for Universally Unique Identifier — a 128-bit number represented as a 36-character string (e.g., `550e8400-e29b-41d4-a716-446655440000`). The function `genrandomuuid()` generates one randomly. The probability of two UUIDs colliding is astronomically low — far safer than sequential integer IDs, especially in distributed systems. **timestamptz** is PostgreSQL's timestamp type that includes timezone information, ensuring consistency regardless of where the server is located.
- To set the UNIQUE constraint: click the gear icon on the column → check "Is Unique"

❒ 3. Table 2: `cardfolders` (Depends on `users`)

COLUMN	TYPE	CONSTRAINTS	DEFAULT	FOREIGN KEY
`id`	uuid	PRIMARY KEY, NOT NULL	`genrandomuuid()`	—
`user_id`	uuid	NOT NULL	—	→ `users.id`, ON DELETE CASCADE
`name`	text	NOT NULL	—	—
`description`	text	nullable	—	—
`created_at`	timestamptz	—	`now()`	—

- To create the foreign key: click the link icon next to `user_id` → select `users` table and `id` column → set ON DELETE to CASCADE

❐ 4. Table 3: `cardsets` (Depends on `users` and `cardfolders`)

COLUMN	TYPE	CONSTRAINTS	DEFAULT	FOREIGN KEY
`id`	uuid	PRIMARY KEY, NOT NULL	`genrandomuuid()`	—
`cardfolder_id`	uuid	NOT NULL	—	→ `cardfolders.id`, ON DELETE CASCADE
`user_id`	uuid	NOT NULL	—	→ `users.id`, ON DELETE CASCADE
`is_public`	bool	NOT NULL	`false`	—
`name`	text	NOT NULL	—	—
`description`	text	nullable	—	—
`created_at`	timestamptz	—	`now()`	—

❐ 5. Table 4: `cards` (Depends on `cardsets` and `users`)

COLUMN	TYPE	CONSTRAINTS	DEFAULT	FOREIGN KEY
id	uuid	PRIMARY KEY, NOT NULL	gen*random*uuid()	—
cardset_id	uuid	NOT NULL	—	→ cardsets.id, ON DELETE CASCADE
front*text*1	text	nullable	—	—
front*au*dio*text_1	text	nullable	—	—
front*au*dio*1	text	nullable	—	—
front_image	text	nullable	—	—
back*text*1	text	nullable	—	—
back*au*dio*text_1	text	nullable	—	—
back*au*dio*1	text	nullable	—	—
back_image	text	nullable	—	—
created_at	timestamptz	—	now()	—
updated_at	timestamptz	nullable	**no default**	—

COLUMN	TYPE	CONSTRAINTS	DEFAULT	FOREIGN KEY
`updat-ed_by`	uuid	nullable	—	→ `users.id` , ON DELETE **SET NULL**

- All eight content fields are nullable because a card might only have some fields populated (e.g., text but no image)
- `updated_at` has **no default** — it starts as null, clearly communicating "never edited"
- `updated_by` uses **SET NULL** (not CASCADE) — if the editor's account is deleted, the card survives with a null reference

❐ 6. The Complete Cascade Chain

- **→ Principle**: Deleting a user triggers a **cascade chain** — their folders are deleted, which deletes their card sets, which deletes their cards. But cards merely *edited* by that user (belonging to other users' sets) survive — only the `updated_by` field is set to null.

❐ 7. Summary of Section III

- Four tables: `users` → `cardfolders` → `cardsets` → `cards`
- Normalisation eliminates data duplication — each fact stored exactly once
- Foreign keys with CASCADE enforce parent-child integrity
- `is_public` on `cardsets` (not folders) controls visibility without contradictions
- `updatedat` + *`updated`* `by` on `cards` create the audit trail
- `ON DELETE SET NULL` on `updated_by` preserves cards when editor accounts are deleted

IV. PROJECT ARCHITECTURE

A. Why Not One File? The Problem of Scale

❐ 1. ✖ The *Monolithic File* Problem

- → **Principle**: With 20+ endpoints, ownership checks, visibility filters, and utility functions, a single file becomes **unreadable and unmaintainable**
- Finding a specific endpoint requires scrolling through hundreds of lines
- Changes to one entity risk accidentally breaking another
- Multiple developers cannot easily work on different parts simultaneously

❐ 2. ✓ The Solution: *Routers* (Mini-Applications)

- → **Principle**: FastAPI's **APIRouter** lets you split endpoints into separate files — each router handles one entity's endpoints, and the main application file includes all routers
- **Context**: A **router** in FastAPI is essentially a mini-application that defines a subset of endpoints. Instead of writing `@app.get("/cardfolders")` in the main file, you write `@router.get("/cardfolders")` in a dedicated file. The router cannot run on its own — it must be plugged into the main FastAPI app using `app.include_router(router)`. This is analogous to chapters in a book: each chapter covers one topic, and the table of contents (main.py) ties them all together.

B. Complete Folder Structure

❒ 1. Directory Layout

```
flashcard-api/
├── .env                          # Supabase credentials (secret,
gitignored)
├── .gitignore                    # Excludes .env, __pycache__
├── main.py                       # Entry point — creates app, includes
routers
├── database.py                   # Supabase connection — exports client
├── models/
│   ├── __init__.py               # Makes directory a Python package
│   ├── user_models.py            # Pydantic models for users
│   ├── cardfolder_models.py      # Pydantic models for folders
│   ├── cardset_models.py         # Pydantic models for card sets
│   └── card_models.py            # Pydantic models for cards
├── routers/
│   ├── __init__.py
│   ├── user_router.py            # Endpoints for user CRUD
│   ├── cardfolder_router.py      # Endpoints for folder CRUD + ownership
│   ├── cardset_router.py         # Endpoints for set CRUD + visibility +
copy
│   └── card_router.py            # Endpoints for card CRUD + audit trail
├── utils/
│   ├── __init__.py
│   └── audio_text.py             # Regex-based audio text generator
```

❒ 2. What `init.py` Files Do

- **Context**: In Python, a directory is just a directory — it has no special meaning to the interpreter. But if you place an empty file called `init.py` inside it, Python treats that directory as a **package**, which means you can import from it using dot notation like `from routers import cardfolder_router`. The `init.py` files can be completely empty — their mere presence is what matters.

C. The Two Foundational Files

❒ 1. `database.py` — The Supabase Connection

- **➜ Principle**: This file reads credentials from the `.env` file, creates a single Supabase client object, and exports it for all routers to import
- Steps performed:

1. Import `os`, `loaddotenv` *from* `dotenv`, `create` `client` and `Client` from `supabase`
2. Call `load_dotenv()` to read the `.env` file into environment variables
3. Pull `SUPABASEURL` *and* `SUPABASE` `KEY` from `os.environ`
4. Create the client: `supabase = create_client(url, key)`

- Every router imports this `supabase` object to talk to the database — there is one shared connection point

❒ 2. `main.py` — The Entry Point

- **➜ Principle**: This file creates the FastAPI application instance, includes all four routers, and defines a root health-check endpoint
- Steps performed:

1. Import `FastAPI`
2. Import all four routers from the `routers/` package
3. Create `app = FastAPI(title=..., description=..., version="2.0.0")`
4. Four calls to `app.include_router(...)` — one per router
5. One root endpoint at `/` returning `"Flashcard API is running"` as a health check

- To start: `uvicorn main:app --reload` — Uvicorn finds the `app` object in `main.py`

❒ 3. Summary of Section IV

- Code split using `APIRouter` — one router per entity

- `models/` for data shapes (Pydantic), `routers/` for endpoint logic, `utils/` for helpers
- `database.py` creates the shared Supabase connection
- `main.py` ties everything together via `include_router`
- Multi-user additions vs. a single-user system: the `userrouter.py` *and* `user` `models.py` files are entirely new

⌘ Directory Structure — Terminal Commands

Commands: Create project folders and empty files

Description: These terminal commands create the entire project skeleton in one step. Three directories are created: `models/` for Pydantic data shapes, `routers/` for endpoint logic, and `utils/` for helper functions. Then all Python files are created as empty placeholders using `touch`, including `init.py` files that mark each directory as a Python package. This structure follows the FastAPI convention of separating concerns: one model file and one router file per database entity.

```
mkdir models routers utils
touch main.py database.py
touch models/__init__.py models/user_models.py models/cardfolder_models.py
models/cardset_models.py models/card_models.py
touch routers/__init__.py routers/user_router.py routers/
cardfolder_router.py routers/cardset_router.py routers/card_router.py
touch utils/__init__.py utils/audio_text.py
```

Notice: The `init.py` files inside `models/`, `routers/`, and `utils/` are essential — without them, Python will not recognise these directories as packages, and import statements like `from models.user_models import UserCreate` will fail. They can remain empty; their mere presence is enough. Also notice that the file creation order does not matter here — these are all empty files. The code inside them will be written in subsequent sections.

⌘ Database Connection — Supabase Client Initialisation

File: `database.py`

Description: This file establishes the single shared connection to the Supabase database that every router uses. It loads the project URL and API key from the `.env` file using `python-dotenv`, then creates a Supabase client instance. This file is imported by every router file via `from database import supabase`, ensuring the entire application shares one connection object rather than creating a new one per request.

```
import os
from dotenv import load_dotenv
from supabase import create_client, Client

load_dotenv()

url: str = os.environ["SUPABASE_URL"]
key: str = os.environ["SUPABASE_KEY"]

supabase: Client = create_client(url, key)
```

Notice: The `load_dotenv()` call must come before `os.environ[...]` — it reads the `.env` file and loads its contents into the environment. Without it, the `os.environ` calls would raise `KeyError` because the variables would not exist. Also note the use of `os.environ[...]` (square brackets) rather than `os.environ.get(...)` — this deliberately crashes at startup if a variable is missing, which is preferable to silently failing later during a database call.

⌘ Application Entry Point — FastAPI App and Router Registration

File: `main.py`

Description: This is the central entry point of the entire API. It creates the FastAPI application instance with metadata (title, description, version) that populates the auto-generated Swagger documentation. It then registers all four routers using `app.include_router()`, which mounts each router's endpoints under their respective URL prefixes. A simple root endpoint at `/` serves as a health check. To start the server: `uvicorn main:app --reload`.

```
from fastapi import FastAPI
from routers import user_router, cardfolder_router, cardset_router,
card_router

app = FastAPI(
    title="Flashcard API",
    description="Multi-user API for managing flashcard folders, sets, and
cards with ownership, public sharing, and copy features",
    version="2.0.0"
)

app.include_router(user_router.router)
app.include_router(cardfolder_router.router)
app.include_router(cardset_router.router)
app.include_router(card_router.router)

@app.get("/")
def root():
    return {"message": "Flashcard API is running"}
```

Notice: The `includerouter` *calls are what connect each router's endpoints to the main application — without them, the endpoints would exist in the router files but would never be reachable. The order of* `include` `router` calls does not affect functionality, but placing more specific route prefixes (like `/cardsets`) before more generic ones is good practice. Also note that `uvicorn main:app` tells Uvicorn to look for an object called `app` inside `main.py` — this is why the variable must be named `app`.

V. PYDANTIC MODELS

A. Why Pydantic Models Are Necessary

❐ 1. The Problem: Unvalidated Input

- ➜ **Principle**: Without validation, malformed data (wrong types, missing fields, extra fields) reaches the database and either causes errors or silently corrupts stored data
- A POST request body might contain a number where a string should be, or omit a required field entirely
- **Context**: When a client sends a POST request (e.g., to create a new flashcard), the data travels as JSON in the request body. JSON is just text — the server needs to parse it and verify it matches the expected structure before doing anything with it. Without explicit validation, the server would blindly attempt to insert whatever arrives into the database.

❐ 2. The Solution: Pydantic Models as Gatekeepers

- ➜ **Principle**: Pydantic models are Python classes that define the **exact shape** of acceptable data — FastAPI uses them to validate every incoming request **before your endpoint code runs**
- ✖ Bad data → automatic **422 Unprocessable Entity** response with a detailed error message listing exactly which fields failed and why
- ✔ Valid data → your code receives a clean, type-safe Python object

B. Three Model Categories: Create, Update, Response

❒ 1. ✔ Create Models — What the Client Sends When Making Something New

- **→ Principle**: Create models **exclude** auto-generated fields (`id`, `created_at`) — the database handles those
- They also **exclude** `user_id` — ownership is determined by the request header, not the body (preventing clients from creating resources "on behalf of" another user)

❒ 2. ✔ Update Models — What Can Be Changed After Creation

- **→ Principle**: Update models define **only the fields that may be modified** — some fields are excluded because they should not be changeable through a simple update
- All fields in an Update model are **optional** — the client may only want to change one field, not all of them
- Excluded fields vary by entity (e.g., you cannot change a card set's owner or move it to a different folder via update)

❒ 3. ✔ Response Models — What the API Sends Back

- **→ Principle**: Response models **include** auto-generated fields (`id`, `createdat`*) and audit fields (*`updated` `at`, `updated_by`) — the client needs to see these
- They represent the complete picture of a resource as it exists in the database

C. Models by Entity

❐ 1. User Models

MODEL	FIELDS	NOTES
`UserCreate`	`username` (str, required), `email` (str, required)	Simplest create model
`UserResponse`	`id` , `username` , `email` , `created_at`	Adds auto-generated fields
`UserUpdate`	*(not implemented)*	Tutorial simplification — no username/email changes

❐ 2. Card Folder Models

MODEL	FIELDS	NOTES
`CardfolderCreate`	`name` (str, required), `description` (str, optional, default None)	✖ No `user_id` — set from header
`CardfolderResponse`	`id` , `userid` , `name` , `description` , `created at`	Client can see who owns the folder

❒ 3. Card Set Models

MODEL	FIELDS	NOTES
`CardsetCreate`	`cardfolderid` *(required)*, `name` *(required)*, `description` *(optional)*, `is public` (default False)	✖ No `user_id` — set from header. Private by default.
`CardsetUpdate`	`name` (optional), `description` (optional), `is_public` (optional)	✖ No `cardfolderid` *(cannot move set)*, ✖ *No* `user id` (cannot transfer ownership)
`CardsetResponse`	`id`, `cardfolderid`, `user id`, `name`, `description`, `ispublic`, `created at`	Full picture including ownership and visibility

❒ 4. Card Models

MODEL	FIELDS	NOTES
`CardCreate`	`cardset_id` (required), 8 content fields (all optional)	Content fields: `fronttext1`, `frontaudiotext1`, `front audio1`, `front image`, and the same four for `back_`
`CardUpdate`	8 content fields only (all optional)	✖ No `cardset_id` (cannot move card to different set)
`CardResponse`	`id`, `cardsetid`, *8 content fields*, `created at`, `updatedat` *(optional)*, `updated by` (optional)	Audit fields are optional — null for brand new cards that have never been edited

❐ 5. Critical Exclusion: Audit Fields Are Never Client-Controlled

- → **Principle**: `updatedat` *and* `updated` `by` appear **only** in `CardResponse`, never in `CardCreate` or `CardUpdate` — the API sets them automatically during updates, preventing anyone from faking their audit trail

D. Simplified Authentication: The `x-user-id` Header

❐ 1. How the API Identifies the Requesting User

- → **Principle**: For this tutorial, the client sends their user ID as an HTTP header called `x-user-id` with every request — FastAPI reads it using the `Header` dependency
- **Context**: In a production system, user identity would be established through proper authentication mechanisms like **JWT tokens** (JSON Web Tokens — encrypted tokens that encode user identity and are cryptographically verifiable), **OAuth** (an open protocol for delegated authorization), or **Supabase Auth** (Supabase's built-in authentication service). The `x-user-id` header approach is a simplification that lets the tutorial focus on the ownership and visibility *logic* without the complexity of authentication infrastructure. The underlying concepts (checking who the requester is, comparing them to the resource owner) remain identical regardless of how you identify the user.

❐ 2. FastAPI's Header Dependency Syntax

- `xuserid: str = Header(...)` — the ellipsis (`...`) means the header is **required**; the request fails with 422 if missing
- `xuserid: str = Header(default=None)` — the header is **optional**; if missing, the value is `None`
- → **Principle (naming convention)**: Python variables use underscores (`xuserid`), but HTTP headers use hyphens (`x-user-id`). FastAPI **automatically converts** between the two.

❐ 3. ✖ This Is *Not* Secure for Production

- Anyone can fake the header — there is no verification that the user ID actually belongs to the person making the request
- The simplification is acceptable for learning because the ownership/visibility *logic* is identical regardless of authentication mechanism

❐ 4. Summary of Section V

- Pydantic models for all four entities, split into Create / Update / Response categories
- `user_id` never appears in Create models — it comes from the header, preventing identity spoofing in request bodies
- `is_public` defaults to `false` — private by default
- Audit trail fields appear only in Response models — the API controls them automatically
- Simplified auth via `x-user-id` header with FastAPI's `Header` dependency

⌘ User Models — Identity Data Shapes

File: `models/user_models.py`

Description: This is the simplest model file, defining two Pydantic classes for user data. `UserCreate` specifies what the client must send when registering (username and email only — no ID or timestamp). `UserResponse` specifies what the API returns, including the auto-generated `id` and `created_at` fields that the database produces. There is no `UserUpdate` model because this tutorial does not implement user profile editing.

```
from pydantic import BaseModel
from datetime import datetime

class UserCreate(BaseModel):
    username: str
    email: str

class UserResponse(BaseModel):
    id: str
    username: str
    email: str
    created_at: datetime
```

Notice: There are only two model classes here — no `UserUpdate`. This is a deliberate design choice: the tutorial does not support editing user profiles. Also notice that `id` and `created_at` appear only in the Response model, never in Create — these are auto-generated by the database and should never be set by the client.

⌘ Card Folder Models — Container Data Shapes

File: `models/cardfolder_models.py`

Description: This model file defines the data shapes for card folders — the top-level organisational containers. `CardfolderCreate` requires only a `name` and an optional `description`; crucially, it excludes `userid` *because ownership is set server-side from the request header, not from the client body.* `CardfolderResponse` *includes all fields the database returns, including the* `user` `id` that was injected during creation. Like users, there is no separate Update model — the create model is reused for updates.

```
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class CardfolderCreate(BaseModel):
    name: str
    description: Optional[str] = None

class CardfolderResponse(BaseModel):
    id: str
    user_id: str
    name: str
    description: Optional[str] = None
    created_at: datetime
```

Notice: The `user_id` field is intentionally absent from `CardfolderCreate` — this is the first example of the pattern where ownership is never trusted from the client body. The `description` field uses `Optional[str] = None`, meaning the client can omit it entirely and it defaults to `None`.

⌘ Card Set Models — Visibility and Partial Update Shapes

File: `models/cardset_models.py`

Description: This model file introduces the most complex model pattern in the API with three distinct classes. `CardsetCreate` includes `cardfolderid` *(linking to the parent folder),* `name`*, optional* `description`*, and* `is` `public` defaulting to `False` (private by default). `CardsetUpdate` is the first Update model — all its fields are `Optional` to support partial updates. `CardsetResponse` returns the full set of fields including the server-assigned `userid` *and* `created` `at`.

```
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class CardsetCreate(BaseModel):
    cardfolder_id: str
    name: str
    description: Optional[str] = None
    is_public: bool = False

class CardsetUpdate(BaseModel):
    name: Optional[str] = None
    description: Optional[str] = None
    is_public: Optional[bool] = None

class CardsetResponse(BaseModel):
    id: str
    cardfolder_id: str
    user_id: str
    name: str
    description: Optional[str] = None
    is_public: bool
    created_at: datetime
```

Notice: This is the first entity with all three model types (Create, Update, Response). Pay attention to `ispublic: bool = False` *in the Create model — this means card sets are private by default, and the user must explicitly set* `is` `public: true` to share them. In the Update model, all fields are `Optional` — this is what enables partial updates where the client only sends the fields they want to change.

⌘ Card Models — Content Fields and Audit Trail Shapes

File: `models/card_models.py`

Description: This is the largest model file, defining data shapes for individual flashcards. Cards have eight content fields (front/back text, audio text, audio URL, and image URL) — all optional because a card might have only text, only images, or any combination. The Update model mirrors the content fields as optional for partial updates. The Response model adds audit trail fields (`updatedat` *and* `updated` `by`) that track when and by whom each card was last modified — these appear only in the Response because they are set automatically by the server.

```python
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class CardCreate(BaseModel):
    cardset_id: str
    front_text_1: Optional[str] = None
    front_audio_text_1: Optional[str] = None
    front_audio_1: Optional[str] = None
    front_image: Optional[str] = None
    back_text_1: Optional[str] = None
    back_audio_text_1: Optional[str] = None
    back_audio_1: Optional[str] = None
    back_image: Optional[str] = None

class CardUpdate(BaseModel):
    front_text_1: Optional[str] = None
    front_audio_text_1: Optional[str] = None
    front_audio_1: Optional[str] = None
    front_image: Optional[str] = None
    back_text_1: Optional[str] = None
    back_audio_text_1: Optional[str] = None
    back_audio_1: Optional[str] = None
    back_image: Optional[str] = None

class CardResponse(BaseModel):
    id: str
    cardset_id: str
    front_text_1: Optional[str] = None
    front_audio_text_1: Optional[str] = None
    front_audio_1: Optional[str] = None
    front_image: Optional[str] = None
    back_text_1: Optional[str] = None
    back_audio_text_1: Optional[str] = None
    back_audio_1: Optional[str] = None
    back_image: Optional[str] = None
    created_at: datetime
    updated_at: Optional[datetime] = None
    updated_by: Optional[str] = None
```

Notice: Every content field in `CardCreate` and `CardUpdate` is `Optional` — this is intentional because a flashcard might use only text, only images, or any combination. Also note that `cardsetid` *appears in* `CardCreate` *but not in* `CardUpdate` *— once a card is created inside a card set, it cannot be moved to a different set. The* `updated` `at` and `updated_by` audit trail fields appear only in `CardResponse`, never in Create or Update models, because the server controls them automatically.

⌘ Header Authentication — FastAPI Header Dependency Syntax

File: Used across all routers (not a standalone file)

Description: This snippet demonstrates the two patterns for extracting the `x-user-id` header that all routers use for authentication. `Header(...)` makes the header mandatory — the request fails with 422 if missing. `Header(default=None)` makes the header optional — allowing anonymous access for public resources. This is not a separate file but a reference for the syntax used throughout every router.

```
from fastapi import Header

# Required header — request fails with 422 if missing:
def my_endpoint(x_user_id: str = Header(...)):

# Optional header — defaults to None if missing:
def my_endpoint(x_user_id: str = Header(default=None)):
```

Notice: The `...` (Ellipsis) in `Header(...)` is Python's built-in Ellipsis object — FastAPI interprets it as "this parameter is required." This is different from `Header()` with no arguments, which would default to `None`. The `x-user-id` header name is automatically converted from the Python parameter name `xuserid` (underscores become hyphens). Write operations use `Header(...)` (mandatory); read operations that allow public access use `Header(default=None)` (optional).

VI. BUILDING EVERY ENDPOINT

A. Overview: 21 Endpoints Across Four Routers

❐ 1. Endpoint Count by Router

ROUTER	ENDPOINTS	COUNT
Users	Register, List All, Get One	3
Folders	Create, Get Mine, Get One, Update, Delete	5
Card Sets	Create, Get Mine, Get Public, Get One, Get By Folder, Update, Delete	7
Cards	Create, Get One, Get By Card Set, Update, Delete, Update History	6
Total		**21** (+ 1 copy endpoint in Section VIII = 22)

B. HTTP Status Codes: The Language of API Responses

❐ 1. Key Status Codes Used in This API

- **Context**: Every HTTP response includes a numeric **status code** that tells the client what happened. Codes in the 200 range mean success. Codes in the 400 range mean the client made an error. Codes in the 500 range mean the server had an internal error.

CODE	MEANING	WHEN USED
200	OK	Successful read, update, or delete
201	Created	Successful creation of a new resource
401	Unauthorized	"We don't know who you are" — the user ID does not correspond to any registered user (**authentication failure**)
403	Forbidden	"We know who you are, but you're not allowed" — the user exists but does not own the resource (**authorisation failure**)
404	Not Found	The requested resource does not exist
409	Conflict	The request format is valid, but it conflicts with current state (e.g., duplicate username)
422	Unprocessable Entity	The request body fails Pydantic validation

- **→ Principle (401 vs. 403)**: The distinction between 401 (authentication) and 403 (authorisation) is critical. 401 means identity is unknown or unverified. 403 means identity is known but permissions are insufficient.

C. Helper Functions: Reusable Ownership Verification

❒ 1. `verifyuser(userid)`

- Queries the `users` table for the given ID
- ✖ If user does not exist → raises **401 Unauthorized**
- Called at the start of any endpoint that requires a known user

❒ 2. `verifyfolderownership(folderid, userid)`

- Fetches the folder by ID
- ✖ If folder does not exist → **404 Not Found**
- ✖ If folder exists but `user_id` does not match → **403 Forbidden**

❒ 3. `verifycardsetownership(cardsetid, userid)`

- Same pattern as folder ownership — 404 or 403

❒ 4. `getcardandverifyownership(cardid, userid)`

- ➙ **Principle**: Cards do not have their own `user_id` — ownership is inherited from the parent card set. This helper performs a **two-step chain**: fetch the card (404 if missing), then verify ownership of its parent card set (403 if not the owner).

D. User Router — 3 Endpoints

❒ 1. `POST /users` — Register a New User

- ➜ **Principle**: Before inserting, the endpoint checks whether the username or email already exists — if either does, it raises **409 Conflict** with a clear message like "Username already taken"
- **Context**: We check explicitly (rather than relying on the database's UNIQUE constraint to throw an error) so we can return a human-readable message. A raw database error would be cryptic and unhelpful to the client.

❒ 2. `GET /users` — List All Users

- Returns all registered users — no authentication required for this tutorial

❒ 3. `GET /users/{userid}` — Get One Specific User

- Returns a single user by their ID — 404 if not found

E. Folder Router — 5 Endpoints

❐ 1. `POST /cardfolders` — Create a Folder (Owner = Requester)

- ➔ **Principle**: Ownership is set server-side — the line `data["userid"] = xuser_id` ensures the folder owner is determined by the header, not the request body. The client cannot claim to create a folder on behalf of another user.
- Calls `verify_user` first

❐ 2. `GET /cardfolders/my` — Get All My Folders

- Filters by `user_id` matching the header — returns only folders owned by the requester

❐ 3. `GET /cardfolders/{folderid}` — Get One Folder

- Returns the folder by ID — 404 if not found

❐ 4. `PUT /cardfolders/{folderid}` — Update a Folder (Owner Only)

- Calls `verifyuser` *then* `verify` `folder_ownership`
- ✖ Non-owner → 403 Forbidden

❐ 5. `DELETE /cardfolders/{folderid}` — Delete a Folder (Owner Only, Cascades)

- Calls `verifyuser` *then* `verify` `folder_ownership`
- Deletion cascades to all card sets inside, which cascades to all cards inside those sets

F. Card Set Router — 7 Endpoints

❒ 1. `POST /cardsets` — Create a Card Set

- → **Principle**: Two verifications before inserting — (1) the requesting user exists, (2) the parent folder exists **and belongs to that user**. You cannot create a card set inside someone else's folder.

❒ 2. `GET /cardsets/my` — Get All My Card Sets

- Returns all card sets where `user_id` matches the header — no visibility filtering needed (you see everything that is yours)

❒ 3. `GET /cardsets/public` — Get All Public Card Sets

- Returns all card sets where `is_public` is `true`
- ✖ No authentication required — anyone can browse public sets

❒ 4. `GET /cardsets/{cardsetid}` — Get One Card Set (Visibility Logic)

- → **Principle**: The `x-user-id` header is **optional** here (`Header(default=None)`) — because a public card set should be viewable by anyone, including anonymous users
- If the card set is public → return it regardless of who is asking
- If the card set is private → check if the requester is the owner
- ✖ Not the owner or no header sent → **403 Forbidden**

❒ 5. `GET /cardsets/folder/{folderid}` — Get Card Sets in a Folder (Filtered)

- Fetches all card sets in the folder, then **filters in Python**: keeps sets that are either public OR owned by the requesting user
- **Context**: Supabase's chained query API does not easily support OR conditions like "where `ispublic` *= true OR* `user` `id` = X." For small to medium datasets, filtering in Python after fetching all results is perfectly acceptable. For very large datasets, you could use a Supabase stored

procedure or the `.or_filter` method.

❐ 6. `PUT /cardsets/{cardsetid}` — Update a Card Set (Owner Only)

- Calls `verifyuser` *then* `verify` `cardset_ownership`
- **→ Principle**: Uses `modeldump(excludeunset=True)` — this is crucial. It means only fields the client **actually sent** get included in the update. Without it, fields the client did not mention would be sent as `None`, overwriting existing data in the database.
- **Context**: `modeldump()` *is a Pydantic method that converts a model instance to a dictionary. The* `exclude` `unset=True` parameter tells it to exclude any field that the client did not explicitly include in their request — as opposed to fields that were explicitly set to `None`. This distinction is important: "I didn't mention the description" (leave it unchanged) is different from "I set the description to null" (clear it).

❐ 7. `DELETE /cardsets/{cardsetid}` — Delete a Card Set (Owner Only, Cascades)

- Owner verification → delete → cascades to all cards in the set

---

G. Card Router — 6 Endpoints

❒ 1. `POST /cards` — Create a Card

- Verifies user exists and that they own the parent card set (prevents User B from sneaking cards into User A's set)
- **→ Principle (audio auto-generation)**: If the client provides `fronttext1` but **not** `frontaudiotext1`*, the API automatically calls* `generate` `audiotext()` *to create a TTS-friendly version (see Section VII). Same for* `back` `text_1`.

❒ 2. `GET /cards/{cardid}` — Get One Card (Visibility Inherited)

- **→ Principle**: Card visibility is determined by the **parent card set's** `is_public` flag. If the set is private and the requester is not the owner → 403 Forbidden. Visibility cascades — making a set private hides all its cards.

❒ 3. `GET /cards/cardset/{cardsetid}` — Get All Cards in a Card Set

- Same visibility check as above — applied to the parent card set before returning any cards

❒ 4. `PUT /cards/{cardid}` — Update a Card (Owner Only + Audit Trail)

- Verifies ownership through the two-step chain (card → parent card set → check owner)
- Handles audio text regeneration if front/back text changes but audio text is not explicitly provided
- **→ Principle (audit trail stamping)**: Two critical lines execute after the update data is assembled:
- `data["updated_at"] = datetime.now(timezone.utc).isoformat()` — stamps the current UTC time as an ISO 8601 string (e.g., `"2025-03-15T14:30:00+00:00"`)
- `data["updatedby"] = xuser_id` — records who made the edit

- **Context**: **UTC** (Coordinated Universal Time) is the global time standard. Using `timezone.utc` ensures every timestamp is in the same reference frame regardless of where the server physically runs. **ISO 8601** is an international standard for date/time representation — the format `YYYY-MM-DDTHH:MM:SS+00:00` is unambiguous and machine-parseable across all programming languages and systems. Both `datetime` and `timezone` come from Python's built-in `datetime` module — no extra packages required.

❐ 5. `DELETE /cards/{cardid}` — Delete a Card (Owner Only)

- Uses the two-step ownership chain: verify card exists, verify parent card set ownership

❐ 6. `GET /cards/cardset/{cardsetid}/updates` — View Audit History

- → **Principle**: Returns a **lightweight view** of all cards in a set, selecting only identification fields and audit fields (`id`, `cardsetid`, `updated` `at`, `updated_by`) — functions as a dashboard showing which cards have been recently modified and by whom
- The SELECT call requests only specific columns rather than all content fields — more efficient for this overview purpose

---

H. Key Patterns Across All Routers — Summary

❐ 1. Authentication Patterns

PATTERN	SYNTAX	USE CASE
Mandatory auth	`Header(...)`	Write operations (create, update, delete)
Optional auth	`Header(default=None)`	Read operations where public access is permitted

❐ 2. Consistent Error Responses

SITUATION	STATUS CODE
Unknown user ID	401
Known user, insufficient permissions	403
Resource not found	404
Duplicate username/email	409
Invalid request body	422

❐ 3. Ownership Enforcement

- → **Principle**: Every single write operation (create, update, delete) across all four routers verifies ownership before proceeding. The pattern is always: verify user → verify ownership → perform action.
- Cards inherit ownership from their parent card set — no direct `user_id` on cards

❐ 4. Visibility Cascade

- → **Principle**: Visibility is controlled at the card set level and cascades to all cards within. There are no per-card visibility flags — if the set is private, all its cards are private.

⌘ User Router — Registration and Retrieval

File: `routers/user_router.py`

Description: This is the simplest router in the API, handling user registration and retrieval with three endpoints. It provides: registering a new user with duplicate username/email checking (409 Conflict), listing all registered users, and fetching a single user by ID. Unlike the other routers, it does not require the `x-user-id` header for authentication — user management is open in this tutorial. It serves as the identity foundation that all other routers depend on for ownership verification.

```
from fastapi import APIRouter, HTTPException
from models.user_models import UserCreate, UserResponse
from database import supabase
from typing import List

router = APIRouter(
    prefix="/users",
    tags=["Users"]
)


# ──────────────────────────────────────
# ENDPOINT 1: Register a new user
# ──────────────────────────────────────
@router.post("/", response_model=UserResponse)
def create_user(user: UserCreate):
    # Check if username already exists
    existing = supabase.table("users").select("id").eq("username",
user.username).execute()
    if existing.data:
        raise HTTPException(status_code=409, detail="Username already
taken")

    # Check if email already exists
    existing_email = supabase.table("users").select("id").eq("email",
user.email).execute()
    if existing_email.data:
        raise HTTPException(status_code=409, detail="Email already
registered")

    data = user.model_dump()
    result = supabase.table("users").insert(data).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to create
user")

    return result.data[0]
```

```
# ─────────────────────────────────────
# ENDPOINT 2: Get all users
# ─────────────────────────────────────
@router.get("/", response_model=List[UserResponse])
def get_all_users():
    result = supabase.table("users").select("*").execute()
    return result.data


# ─────────────────────────────────────
# ENDPOINT 3: Get one user by ID
# ─────────────────────────────────────
@router.get("/{user_id}", response_model=UserResponse)
def get_user(user_id: str):
    result = supabase.table("users").select("*").eq("id",
user_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail="User not found")

    return result.data[0]
```

Notice: The duplicate checks for username and email are performed as two separate queries before insertion rather than relying on database-level UNIQUE constraints to throw errors. This is deliberate — it allows the API to return a human-readable message like "Username already taken" instead of a cryptic database error. Also note that this router has no `Header(...)` dependency at all, making it the only router without authentication.

⌘ Card Folder Router — Ownership-Protected CRUD

File: `routers/cardfolder_router.py`

Description: This router manages card folders — the top-level organisational containers in the hierarchy. It provides five endpoints: create a folder (ownership set server-side via the header), list all folders owned by the requester, get a single folder by ID, update a folder (owner only), and delete a folder (owner only, cascades to child card sets and their cards). It introduces two reusable helper functions (`verifyuser` *and* `verify` `folder_ownership`) that establish the authentication-then-authorisation pattern used throughout the API.

```python
from fastapi import APIRouter, HTTPException, Header
from models.cardfolder_models import CardfolderCreate, CardfolderResponse
from database import supabase
from typing import List

router = APIRouter(
    prefix="/cardfolders",
    tags=["Card Folders"]
)


# ──────────────────────────────────────────────────────────────
# HELPER: Verify the requesting user exists in the database
# ──────────────────────────────────────────────────────────────
def verify_user(user_id: str):
    result = supabase.table("users").select("id").eq("id", user_id).execute()
    if not result.data:
        raise HTTPException(status_code=401, detail="User not found — invalid x-user-id header")


# ──────────────────────────────────────────────────────────────
# HELPER: Verify the requesting user owns the given folder
# ──────────────────────────────────────────────────────────────
def verify_folder_ownership(folder_id: str, user_id: str):
    result = supabase.table("cardfolders").select("*").eq("id", folder_id).execute()
    if not result.data:
        raise HTTPException(status_code=404, detail="Folder not found")
    folder = result.data[0]
    if folder["user_id"] != user_id:
        raise HTTPException(status_code=403, detail="You do not own this folder")
    return folder


# ──────────────────────────────────────
# ENDPOINT 1: Create a new folder
```

```
# ──────────────────────────────────────
@router.post("/", response_model=CardfolderResponse)
def create_cardfolder(folder: CardfolderCreate, x_user_id: str =
Header(...)):
    verify_user(x_user_id)

    data = folder.model_dump()
    data["user_id"] = x_user_id  # Set the owner to the requesting user

    result = supabase.table("cardfolders").insert(data).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to create
folder")

    return result.data[0]


# ──────────────────────────────────────────
# ENDPOINT 2: Get all folders owned by me
# ──────────────────────────────────────────
@router.get("/my", response_model=List[CardfolderResponse])
def get_my_cardfolders(x_user_id: str = Header(...)):
    verify_user(x_user_id)
    result = supabase.table("cardfolders").select("*").eq("user_id",
x_user_id).execute()
    return result.data


# ──────────────────────────────────────
# ENDPOINT 3: Get one folder by ID
# ──────────────────────────────────────
@router.get("/{folder_id}", response_model=CardfolderResponse)
def get_cardfolder(folder_id: str):
    result = supabase.table("cardfolders").select("*").eq("id",
folder_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail="Folder not found")
```

```
    return result.data[0]


# ──────────────────────────────────────────
# ENDPOINT 4: Update a folder (owner only)
# ──────────────────────────────────────────
@router.put("/{folder_id}", response_model=CardfolderResponse)
def update_cardfolder(folder_id: str, folder: CardfolderCreate, x_user_id:
str = Header(...)):
    verify_user(x_user_id)
    verify_folder_ownership(folder_id, x_user_id)

    data = folder.model_dump()
    result = supabase.table("cardfolders").update(data).eq("id",
folder_id).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to update
folder")

    return result.data[0]


# ──────────────────────────────────────────
# ENDPOINT 5: Delete a folder (owner only)
# ──────────────────────────────────────────
@router.delete("/{folder_id}")
def delete_cardfolder(folder_id: str, x_user_id: str = Header(...)):
    verify_user(x_user_id)
    verify_folder_ownership(folder_id, x_user_id)

    result = supabase.table("cardfolders").delete().eq("id",
folder_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail="Folder not found")

    return {"message": "Folder deleted successfully", "deleted":
```

```
result.data[0]}
```

Notice: Pay attention to the line `data["userid"] = xuserid` *in the create endpoint — this is where ownership is enforced server-side. The client's request body (via the Pydantic model) never contains* `user` `id`; it is injected from the header after validation. This prevents any client from creating a folder "on behalf of" another user. Also note that `verifyfolderownership` returns the folder data if ownership passes — this avoids a redundant database query when the endpoint needs the folder object after verification.

⌘ Card Set Router — Visibility and Ownership Logic

File: `routers/cardset_router.py`

Description: This is the most complex router, with seven endpoints handling creation, retrieval, update, and deletion of card sets. It introduces the `ispublic` *visibility system: public sets are viewable by anyone, private sets only by their owner. Key endpoints include: create a set inside an owned folder, get all your own sets, browse all public sets, get a single set (with visibility logic), get sets in a folder (filtered by visibility), update (owner only, using* `model` `dump(exclude_unset=True)` for partial updates), and delete (owner only, cascading to all child cards).

```
from fastapi import APIRouter, HTTPException, Header
from models.cardset_models import CardsetCreate, CardsetUpdate,
CardsetResponse
from database import supabase
from typing import List

router = APIRouter(
    prefix="/cardsets",
    tags=["Card Sets"]
)


# ──────────────────────────────────────────────────────────────
# HELPER: Verify the requesting user exists in the database
# ──────────────────────────────────────────────────────────────
def verify_user(user_id: str):
    result = supabase.table("users").select("id").eq("id",
user_id).execute()
    if not result.data:
        raise HTTPException(status_code=401, detail="User not found —
invalid x-user-id header")


# ──────────────────────────────────────────────────────────────
# HELPER: Verify the requesting user owns the given card set
# Returns the card set data if ownership is confirmed
# ──────────────────────────────────────────────────────────────
def verify_ownership(cardset_id: str, user_id: str):
    result = supabase.table("cardsets").select("*").eq("id",
cardset_id).execute()
    if not result.data:
        raise HTTPException(status_code=404, detail="Card set not found")
    cardset = result.data[0]
    if cardset["user_id"] != user_id:
        raise HTTPException(status_code=403, detail="You do not own this
card set")
    return cardset
```

```
# ──────────────────────────────────────
# ENDPOINT 1: Create a new card set
# ──────────────────────────────────────
@router.post("/", response_model=CardsetResponse)
def create_cardset(cardset: CardsetCreate, x_user_id: str = Header(...)):
    verify_user(x_user_id)

    # Verify the parent folder exists and belongs to this user
    folder_check = supabase.table("cardfolders").select("*").eq("id",
cardset.cardfolder_id).execute()
    if not folder_check.data:
        raise HTTPException(status_code=404, detail="Parent folder not
found")
    if folder_check.data[0]["user_id"] != x_user_id:
        raise HTTPException(status_code=403, detail="You do not own the
parent folder")

    data = cardset.model_dump()
    data["user_id"] = x_user_id  # Set the owner to the requesting user

    result = supabase.table("cardsets").insert(data).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to create card
set")

    return result.data[0]


# ──────────────────────────────────────────
# ENDPOINT 2: Get all card sets owned by me
# ──────────────────────────────────────────
@router.get("/my", response_model=List[CardsetResponse])
def get_my_cardsets(x_user_id: str = Header(...)):
    verify_user(x_user_id)
    result = supabase.table("cardsets").select("*").eq("user_id",
x_user_id).execute()
    return result.data
```

```
# ──────────────────────────────────────────
# ENDPOINT 3: Get all public card sets
# ──────────────────────────────────────────
@router.get("/public", response_model=List[CardsetResponse])
def get_public_cardsets():
    result = supabase.table("cardsets").select("*").eq("is_public",
True).execute()
    return result.data


# ──────────────────────────────────────────────────────────
# ENDPOINT 4: Get one card set by ID (visibility enforced)
# ──────────────────────────────────────────────────────────
@router.get("/{cardset_id}", response_model=CardsetResponse)
def get_cardset(cardset_id: str, x_user_id: str = Header(default=None)):
    result = supabase.table("cardsets").select("*").eq("id",
cardset_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail="Card set not found")

    cardset = result.data[0]

    # If the set is private, only the owner can see it
    if not cardset["is_public"] and cardset["user_id"] != x_user_id:
        raise HTTPException(status_code=403, detail="This card set is
private")

    return cardset


# ──────────────────────────────────────────────────────────────────
# ENDPOINT 5: Get card sets in a folder (visibility enforced)
# ──────────────────────────────────────────────────────────────────
@router.get("/folder/{folder_id}", response_model=List[CardsetResponse])
def get_cardsets_by_folder(folder_id: str, x_user_id: str =
Header(default=None)):
    result = supabase.table("cardsets").select("*").eq("cardfolder_id",
```

```
folder_id).execute()

    # Filter: show public sets + sets owned by the requesting user
    visible = []
    for cs in result.data:
        if cs["is_public"] or cs["user_id"] == x_user_id:
            visible.append(cs)

    return visible


# ─────────────────────────────────────────────
# ENDPOINT 6: Update a card set (owner only)
# ─────────────────────────────────────────────
@router.put("/{cardset_id}", response_model=CardsetResponse)
def update_cardset(cardset_id: str, cardset: CardsetUpdate, x_user_id: str
= Header(...)):
    verify_user(x_user_id)
    verify_ownership(cardset_id, x_user_id)

    data = cardset.model_dump(exclude_unset=True)

    if not data:
        raise HTTPException(status_code=400, detail="No fields to update")

    result = supabase.table("cardsets").update(data).eq("id",
cardset_id).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to update card
set")

    return result.data[0]


# ─────────────────────────────────────────────
# ENDPOINT 7: Delete a card set (owner only)
# ─────────────────────────────────────────────
@router.delete("/{cardset_id}")
```

```
def delete_cardset(cardset_id: str, x_user_id: str = Header(...)):
    verify_user(x_user_id)
    verify_ownership(cardset_id, x_user_id)

    result = supabase.table("cardsets").delete().eq("id",
cardset_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail="Card set not found")

    return {"message": "Card set deleted successfully", "deleted":
result.data[0]}
```

Notice: Two critical patterns to study here. First, Endpoint 4 (`GET /{cardset`*`id}`*) *uses `Header(default=None)` instead of `Header(...)` — making authentication optional because public sets should be viewable by anyone, including anonymous users. Second, the update endpoint uses `model` `dump(exclude`*`unset=True)`* — this ensures only fields the client actually sent are included in the update. Without* `exclude` `unset=True`, omitted fields would be sent as `None` to the database, overwriting existing data. The distinction between "not mentioned" and "explicitly set to null" is crucial.

⌘ Card Router — CRUD with Inherited Ownership and Audit Trail

File: `routers/card_router.py`

Description: This router manages individual flashcards with six endpoints. Cards do not have their own `userid` *— ownership is inherited from the parent card set through a two-step verification chain. The router integrates the audio text auto-generation utility (Section VII): when a card is created or updated with display text but no audio text, the API automatically generates a TTS-friendly version. The update endpoint writes audit trail fields (*`updated``at` with UTC timestamp, `updated_by` with the user ID), and a dedicated endpoint provides a lightweight dashboard view of modification history.

```
from fastapi import APIRouter, HTTPException, Header
from models.card_models import CardCreate, CardUpdate, CardResponse
from database import supabase
from utils.audio_text import generate_audio_text
from typing import List
from datetime import datetime, timezone

router = APIRouter(
    prefix="/cards",
    tags=["Cards"]
)


# ──────────────────────────────────────────────────────────────
# HELPER: Verify user exists
# ──────────────────────────────────────────────────────────────
def verify_user(user_id: str):
    result = supabase.table("users").select("id").eq("id",
user_id).execute()
    if not result.data:
        raise HTTPException(status_code=401, detail="User not found —
invalid x-user-id header")


# ──────────────────────────────────────────────────────────────
# HELPER: Verify the requesting user owns the card set
# that a card belongs to (or will belong to)
# ──────────────────────────────────────────────────────────────
def verify_cardset_ownership(cardset_id: str, user_id: str):
    result = supabase.table("cardsets").select("*").eq("id",
cardset_id).execute()
    if not result.data:
        raise HTTPException(status_code=404, detail="Parent card set not
found")
    if result.data[0]["user_id"] != user_id:
        raise HTTPException(status_code=403, detail="You do not own the
parent card set")
    return result.data[0]
```

```
# ──────────────────────────────────────────────────────────────
# HELPER: Given a card ID, look up which cardset it belongs to
# and verify ownership. Returns the card data.
# ──────────────────────────────────────────────────────────────
def get_card_and_verify_ownership(card_id: str, user_id: str):
    card_result = supabase.table("cards").select("*").eq("id",
card_id).execute()
    if not card_result.data:
        raise HTTPException(status_code=404, detail="Card not found")
    card = card_result.data[0]
    verify_cardset_ownership(card["cardset_id"], user_id)
    return card


# ──────────────────────────────────────
# ENDPOINT 1: Create a new card
# ──────────────────────────────────────
@router.post("/", response_model=CardResponse)
def create_card(card: CardCreate, x_user_id: str = Header(...)):
    verify_user(x_user_id)
    verify_cardset_ownership(card.cardset_id, x_user_id)

    data = card.model_dump()

    # Auto-generate audio text if not provided
    if data.get("front_text_1") and not data.get("front_audio_text_1"):
        data["front_audio_text_1"] =
generate_audio_text(data["front_text_1"])

    if data.get("back_text_1") and not data.get("back_audio_text_1"):
        data["back_audio_text_1"] =
generate_audio_text(data["back_text_1"])

    result = supabase.table("cards").insert(data).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to create
card")
```

```
    return result.data[0]


# ──────────────────────────────────────
# ENDPOINT 2: Get one card by ID
# ──────────────────────────────────────
@router.get("/{card_id}", response_model=CardResponse)
def get_card(card_id: str, x_user_id: str = Header(default=None)):
    card_result = supabase.table("cards").select("*").eq("id",
card_id).execute()
    if not card_result.data:
        raise HTTPException(status_code=404, detail="Card not found")

    card = card_result.data[0]

    # Check visibility of the parent card set
    cardset = supabase.table("cardsets").select("*").eq("id",
card["cardset_id"]).execute()
    if cardset.data:
        cs = cardset.data[0]
        if not cs["is_public"] and cs["user_id"] != x_user_id:
            raise HTTPException(status_code=403, detail="This card belongs
to a private card set")

    return card


# ────────────────────────────────────────────────────────────────
# ENDPOINT 3: Get all cards in a card set (visibility enforced)
# ────────────────────────────────────────────────────────────────
@router.get("/cardset/{cardset_id}", response_model=List[CardResponse])
def get_cards_by_cardset(cardset_id: str, x_user_id: str =
Header(default=None)):
    # Check that the card set exists and is visible
    cs_result = supabase.table("cardsets").select("*").eq("id",
cardset_id).execute()
    if not cs_result.data:
        raise HTTPException(status_code=404, detail="Card set not found")
```

```
    cs = cs_result.data[0]
    if not cs["is_public"] and cs["user_id"] != x_user_id:
        raise HTTPException(status_code=403, detail="This card set is
private")

    result = supabase.table("cards").select("*").eq("cardset_id",
cardset_id).execute()
    return result.data


# ──────────────────────────────────────────────────────────────────
# ENDPOINT 4: Update a card (owner only, writes audit fields)
# ──────────────────────────────────────────────────────────────────
@router.put("/{card_id}", response_model=CardResponse)
def update_card(card_id: str, card: CardUpdate, x_user_id: str =
Header(...)):
    verify_user(x_user_id)
    get_card_and_verify_ownership(card_id, x_user_id)

    data = card.model_dump(exclude_unset=True)

    # Regenerate audio text if display text changed but audio text not
explicitly set
    if "front_text_1" in data and "front_audio_text_1" not in data:
        if data["front_text_1"] is not None:
            data["front_audio_text_1"] =
generate_audio_text(data["front_text_1"])
        else:
            data["front_audio_text_1"] = None

    if "back_text_1" in data and "back_audio_text_1" not in data:
        if data["back_text_1"] is not None:
            data["back_audio_text_1"] =
generate_audio_text(data["back_text_1"])
        else:
            data["back_audio_text_1"] = None

    if not data:
```

```
        raise HTTPException(status_code=400, detail="No fields to update")

    # ── AUDIT TRAIL: Write update timestamp and user ──
    data["updated_at"] = datetime.now(timezone.utc).isoformat()
    data["updated_by"] = x_user_id

    result = supabase.table("cards").update(data).eq("id",
card_id).execute()

    if not result.data:
        raise HTTPException(status_code=400, detail="Failed to update
card")

    return result.data[0]


# ──────────────────────────────────────────────
# ENDPOINT 5: Delete a card (owner only)
# ──────────────────────────────────────────────
@router.delete("/{card_id}")
def delete_card(card_id: str, x_user_id: str = Header(...)):
    verify_user(x_user_id)
    get_card_and_verify_ownership(card_id, x_user_id)

    result = supabase.table("cards").delete().eq("id", card_id).execute()

    if not result.data:
        raise HTTPException(status_code=404, detail="Card not found")

    return {"message": "Card deleted successfully", "deleted":
result.data[0]}


# ──────────────────────────────────────────────────────────────────
# ENDPOINT 6: Get update history for all cards in a card set
# Shows each card's last update timestamp and who updated it
# ──────────────────────────────────────────────────────────────────
@router.get("/cardset/{cardset_id}/updates")
def get_card_updates(cardset_id: str, x_user_id: str =
```

```
Header(default=None)):
    # Visibility check
    cs_result = supabase.table("cardsets").select("*").eq("id",
cardset_id).execute()
    if not cs_result.data:
        raise HTTPException(status_code=404, detail="Card set not found")

    cs = cs_result.data[0]
    if not cs["is_public"] and cs["user_id"] != x_user_id:
        raise HTTPException(status_code=403, detail="This card set is
private")

    result = supabase.table("cards").select(
        "id, front_text_1, back_text_1, created_at, updated_at, updated_by"
    ).eq("cardset_id", cardset_id).execute()

    return result.data
```

Notice: The `getcardandverifyownership` helper performs a two-step chain: fetch the card (404 if missing), then verify ownership of the parent card set (403 if not the owner). This pattern is necessary because cards have no direct `user_id` column — ownership is always inherited. Also pay attention to the audit trail stamping in the update endpoint: `datetime.now(timezone.utc).isoformat()` produces a UTC timestamp in ISO 8601 format (e.g., `"2025-03-15T14:30:00+00:00"`), ensuring consistent timestamps regardless of server location.

VII. AUDIO TEXT GENERATION WITH REGULAR EXPRESSIONS

A. The Problem: Display Text vs. Speech Text

❐ 1. Why Display Text Cannot Be Sent Directly to Text-to-Speech

- **→ Principle**: Flashcard display text contains visual formatting elements (parenthetical translations, square bracket grammar notes, numbering) that produce **unnatural, garbled output** when read aloud by a text-to-speech engine
- **Context**: **TTS (Text-to-Speech)** engines are software systems that convert written text into spoken audio. They read text literally — every character, every punctuation mark. A flashcard displaying `El gato (the cat) [m.]` is visually clear to a reader: "El gato" is the word, "(the cat)" is the translation hint, "[m.]" means masculine gender. But a TTS engine would read it as: "El gato open parenthesis the cat close parenthesis open bracket m period close bracket" — which is unintelligible and useless for language learning.
- The goal: transform `El gato (the cat) [m.]` into `El gato` — clean, natural text ready for speech

---

B. The Tool: `re.sub()` — Python's Regex Substitution Function

❒ 1. How `re.sub()` Works

- **Context**: **Regular expressions (regex)** are a powerful pattern-matching language used across virtually all programming languages. Instead of searching for a specific string (like "find the word 'cat'"), a regex describes a *pattern* (like "find anything between square brackets"). Python's `re` module provides regex functions, and `re.sub()` is the substitution function.
- Syntax: `re.sub(pattern, replacement, string)`
- `pattern` — the regex pattern describing what to find
- `replacement` — what to replace each match with (often an empty string `""` to delete the match)
- `string` — the text to search within
- Returns a new string with all matches replaced

❒ 2. Greedy vs. Non-Greedy Matching

- → **Principle**: The `?` quantifier makes a regex pattern **non-greedy** (match as *few* characters as possible), which is critical when multiple bracketed or parenthesised sections exist in the same string
- **Context**: By default, regex patterns are "greedy" — they match as *many* characters as possible. Consider the text `[one] word [two]`. A greedy pattern like `\[.\]` *would match from the* first `[` *all the way to the* last `]`, *capturing* `[one] word [two]` *as a single match — the word between them would be deleted. A non-greedy pattern* `\[.` `?\]` matches each bracket pair *individually*: first `[one]`, then `[two]`, preserving the word between them.

C. The Five Transformation Rules (Applied in Order)

❐ 1. Rule 1: Remove Square Bracket Content

- Pattern: `\[.*?\]`
- Replacement: `""` (empty string — delete)
- Example: `El gato [m.]` → `El gato`
- Removes grammar notes, pronunciation guides, and other annotations

❐ 2. Rule 2: Remove Parenthetical Content

- Pattern: `\(.*?\)`
- Replacement: `""` (empty string — delete)
- Example: `El gato (the cat)` → `El gato`
- Removes translation hints and explanatory notes
- Uses non-greedy matching for the same reason as Rule 1

❐ 3. Rule 3: Simplify Slash Alternatives (Keep First Form Only)

- **Context**: Language flashcards sometimes show alternatives separated by a slash — e.g., `rojo/a` means the word can be `rojo` (masculine) or `roja` (feminine). For audio, we want only the first form.
- Pattern uses a **capture group**: parentheses in the regex `(\w+)/\w+` capture the first word
- Replacement: `\1` (backslash one) — a **backreference** meaning "put back whatever was in the first capture group"
- **Context**: In Python, the replacement is written as `r'\1'`. The `r` prefix makes it a **raw string**, which tells Python not to interpret the backslash as an escape character. Without `r`, Python would interpret `\1` as a special character rather than passing it to the regex engine.
- Example: `rojo/a` → `rojo`

❐ 4. Rule 4: Remove Leading Numbers

- Pattern matches digits followed by a dot at the start of the string
- Example: `3. El gato` → `El gato`
- Removes card numbering that is irrelevant for audio playback

❐ 5. Rule 5: Clean Up Whitespace (Must Be Last)

- **→ Principle**: This rule **must be applied last** because all previous rules leave behind gaps (extra spaces) where content was removed
- Collapses multiple consecutive spaces into a single space
- Strips leading and trailing whitespace
- If run earlier, the gaps created by subsequent rules would remain uncleaned

---

D. Complete Walkthrough Example

❐ 1. Input → Output Through All Five Rules

STEP	TEXT
Input	`3. El gato rojo/a (the red cat) [m./f.]`
After Rule 1 (brackets)	`3. El gato rojo/a (the red cat)`
After Rule 2 (parentheses)	`3. El gato rojo/a`
After Rule 3 (slash)	`3. El gato rojo`
After Rule 4 (numbering)	`El gato rojo`
After Rule 5 (whitespace)	`El gato rojo`

- **Final result**: Clean, natural text ready for TTS

E. Integration with the Card Router

❐ 1. When Auto-Generation Triggers

- **→ Principle**: The function is called **automatically** in the card router when the client provides display text (`front*text*1` or `back*text*1`) but does **not** provide the corresponding audio text field
- If the client explicitly provides `front*audio*text_1`, that value is used and auto-generation is skipped — the user's explicit choice takes priority

❐ 2. Extensibility

- Adding a new rule is straightforward: add another `text = re.sub(pattern, replacement, text)` line at the appropriate position in the sequence (cleanup rules should remain at the end)

❐ 3. Summary of Section VII

- Five regex rules applied in strict order: remove brackets, remove parentheses, simplify slashes, remove numbering, clean whitespace
- Non-greedy matching (`?`) prevents overmatching when multiple bracket/parenthesis pairs exist
- Capture groups and backreferences enable keeping part of a match while discarding the rest
- Whitespace cleanup must be last to catch gaps left by all preceding rules
- Auto-generation is automatic but overridable — the client can always provide their own audio text

⌘ Audio Text Generator — Regex Transformation Engine

File: `utils/audio_text.py`

Description: This is the utility module that transforms flashcard display text into clean, speech-friendly text for TTS (Text-to-Speech) engines. It serves as a helper function called automatically by the card router whenever a card is created or updated with display text but without an explicit audio text override. The function applies five regex substitution rules in strict sequence: remove square bracket annotations, remove parenthetical hints, simplify slash alternatives to the first form, strip leading numbering, and collapse excess whitespace.

```
import re


def generate_audio_text(display_text: str) -> str:
    """
    Transform display text into audio-friendly text by removing
    annotations, hints, and formatting that would sound unnatural
    when read aloud by a text-to-speech engine.

    Rules applied in order:
    1. Remove content in square brackets: [m.] [noun] [informal]
    2. Remove content in parentheses: (the cat) (lit. house)
    3. Simplify slash alternatives to first word: rojo/a → rojo
    4. Remove leading numbering: 1. 2. 3.
    5. Clean up whitespace
    """
    text = display_text

    # Rule 1: Remove square bracket content
    # \[   matches a literal opening bracket
    # .*?  matches any characters, non-greedy (as few as possible)
    # \]   matches a literal closing bracket
    text = re.sub(r'\[.*?\]', '', text)

    # Rule 2: Remove parenthetical content
    # \(   matches a literal opening parenthesis
    # .*?  matches any characters, non-greedy
    # \)   matches a literal closing parenthesis
    text = re.sub(r'\(.*?\)', '', text)

    # Rule 3: Simplify slash alternatives — keep only first word
    # (\w+)  captures one or more word characters (the part we keep)
    # /      matches a literal slash
    # \w+    matches one or more word characters (the part we discard)
    # \1     in the replacement refers to the first captured group
    text = re.sub(r'(\w+)/\w+', r'\1', text)

    # Rule 4: Remove leading numbering like "1. " or "23. "
    # ^      anchors to the start of the string
```

```
    # \d+    matches one or more digits
    # \.     matches a literal dot
    # \s*    matches zero or more whitespace characters
    text = re.sub(r'^\d+\.\s*', '', text)

    # Rule 5: Collapse multiple spaces into one
    # \s+    matches one or more whitespace characters
    text = re.sub(r'\s+', ' ', text)

    # Final strip to remove leading/trailing whitespace
    text = text.strip()

    return text
```

Notice: The order of the five rules is critical — Rule 5 (whitespace cleanup) must always be last because every preceding rule leaves behind extra spaces where removed content used to be. Also note the non-greedy quantifier (`*?`) in Rules 1 and 2: without it, the pattern would match from the first opening bracket to the last closing bracket in the string, accidentally deleting everything between separate bracket pairs.

VIII. PUBLIC SHARING AND THE COPY FEATURE

A. Why Copying Is Necessary

❐ 1. The Problem: Public Content Is Read-Only to Non-Owners

- **→ Principle**: A user who discovers a public card set cannot edit it (they are not the owner), and even if they could, their changes would affect every other user viewing the same set
- **Context**: Imagine User A is a Spanish teacher who spent hours building a card set called "DELE B2 Vocabulary" with 200 carefully crafted cards — audio text, image URLs, everything perfect. They mark it public so students can find it. User B, a student, discovers it and wants to study from it — but also wants to add personal notes, tweak translations, remove cards they already know. The student cannot modify the teacher's original. And even if permission were granted, those modifications would affect every other student using the same public set.

❐ 2. The Solution: Deep Copy — A Complete, Independent Duplicate

- **→ Principle**: The copy feature creates a **complete, independent clone** of a card set and all its cards into the requesting user's account — the copy belongs entirely to them, and the original is completely unaffected

B. The Five-Step Copy Process

❐ 1. Step-by-Step Sequence

STEP	ACTION	VERIFICATION
1	Verify source card set exists and is accessible	Must be public OR owned by the requester (you can copy your own sets to create variations)
2	User specifies target folder via `targetfolderid` query parameter	Folder must exist AND belong to the requesting user
3	Create new card set record	Name = "Copy of [original name]", owner = requester, folder = target folder, `is_public` = false (private by default)
4	Fetch all cards from the source card set	—
5	Duplicate each card into the new card set	Content preserved exactly; IDs, timestamps, and audit fields start fresh

❐ 2. What Gets Copied vs. What Starts Fresh

❐ a. ✓ *Preserved* Exactly

- All eight content fields: `fronttext1`, `frontaudiotext1`, `front` `audio1`, `front` `image`, `backtext1`, `backaudiotext1`, `back` `audio1`, `back` `image`

❒ b. ✖ *Not Copied* — Generated Fresh

FIELD	WHY
`id`	Each new card gets a fresh UUID
`cardset_id`	Points to the **new** card set, not the original
`created_at`	Set to current time by the database — these are new records
`updated_at`	Null — the copied card has never been edit-ed, only created
`updated_by`	Null — no edits have occurred

C. The Endpoint

❒ 1. URL Pattern and Method

- `POST /cardsets/{cardsetid}/copy?targetfolder_id=<uuid>`
- The `{cardset_id}` in the URL = the source card set to copy
- The `targetfolderid` = query parameter specifying which of the requester's folders to place the copy in
- Uses POST because this operation **creates** new data

❒ 2. Defensive Programming: `.get()` vs. Bracket Access

- **→ Principle**: When reading fields from source cards, the code uses `card.get("fieldname")` *instead of* `card["field` `name"]` — if a field does not exist or is not populated, `.get()` returns `None` gracefully, while bracket access would crash with a `KeyError`
- **Context**: This is a defensive programming pattern. Not every card will have all eight content fields filled in — some might only have text, no images. Using `.get()` handles these partial-data edge cases without special-casing each field.

❒ 3. Performance: Batch Insert

- **→ Principle**: Instead of inserting cards one at a time in a loop (one database call per card), all new cards are collected in a list and inserted with a **single** database call — Supabase's `insert()` method accepts a list of dictionaries
- Copying 200 cards = one network call instead of 200
- Dramatically faster, especially over network connections with latency

❒ 4. Return Value

- The endpoint returns the new card set record — the client receives the ID of their copy and can use `GET /cards/cardset/{newcardsetid}` to see all copied cards

D. Full End-to-End Test Scenario

❒ 1. Setup: Two Users, Two Folders

1. `POST /users` — register `teacher_ana` → save her user ID
2. `POST /users` — register `student_bob` → save his user ID
3. `POST /cardfolders` (header: teacher's ID) — create folder "Languages" → save folder ID
4. `POST /cardfolders` (header: student's ID) — create folder "My Studies" → save folder ID

❒ 2. Teacher Creates Public Content

1. `POST /cardsets` (header: teacher's ID) — create "Spanish Basics" in Languages folder, `is_public: true` → save card set ID
2. `POST /cards` (header: teacher's ID) — create card with `front`*`text`*`1: "El gato (the cat) [m.]"`, `back`*`text`*`1: "The cat"` → observe `front`*`audio`*`text_1` auto-generated as `"El gato"`

❒ 3. Student Discovers and Copies

1. `GET /cardsets/public` (no header needed) — student sees "Spanish Basics" listed
2. `POST /cardsets/{id}/copy?target`*`folder`*`id={student`*`folder`*`id}` (header: student's ID) → response: "Copy of Spanish Basics," owned by student, private

❒ 4. Student Personalises Their Copy

1. `GET /cardsets/my` (header: student's ID) — "Copy of Spanish Basics" appears
2. `GET /cards/cardset/{new`*`set`*`id}` — see all copied cards, pick one card ID
3. `PUT /cards/{card`*`id}`* *(header: student's ID) — change `front` `text1` to `"El gato gordo (the fat cat)"` → `updated` `at`* = current timestamp, `updated`*`by`* = *student's ID,* `front` `audio`*`text1`* auto-regenerated as `"El

`gato gordo"`

❐ 5. Verifying Isolation and Ownership

1. `GET /cards/cardset/{originalsetid}` — teacher's cards are completely untouched
2. `PUT /cards/{teachercardid}` (header: student's ID) → **403 Forbidden**: "You do not own the parent card set" — ownership system enforced

---

E. Summary of Section VIII

- The copy endpoint performs a **deep copy**: new card set + all cards duplicated
- Content is preserved exactly; metadata (IDs, timestamps, audit fields) starts fresh
- Only public sets or your own sets can be copied
- Target folder must belong to the requesting user
- Batch insert for performance
- The copy is **completely independent** from the original — edits to one never affect the other

⌘ Copy Feature — Public Card Set Duplication

File: `routers/cardset_router.py` **(addition)**

Description: This code extends the card set router with the deep copy endpoint — the 22nd and final endpoint of the API. It allows any user to duplicate a public card set (or their own private set) into one of their own folders, creating a completely independent clone of both the set metadata and all its cards. The implementation follows a five-step process: verify the source set is accessible (public or owned by requester), verify the target folder belongs to the requester, create a new card set owned by the requester, fetch all cards from the source, and batch-insert duplicated cards into the new set.

Import to add at the top alongside existing imports:

```
from utils.audio_text import generate_audio_text
```

Copy endpoint to add at the bottom of the file:

```
# ──────────────────────────────────────────────────────────────────
# ENDPOINT 8: Copy a public card set into your own account
# ──────────────────────────────────────────────────────────────────
@router.post("/{cardset_id}/copy", response_model=CardsetResponse)
def copy_cardset(cardset_id: str, target_folder_id: str, x_user_id: str =
Header(...)):
    verify_user(x_user_id)

    # Step 1: Fetch the source card set and verify it is accessible
    source_result = supabase.table("cardsets").select("*").eq("id",
cardset_id).execute()
    if not source_result.data:
        raise HTTPException(status_code=404, detail="Card set not found")

    source = source_result.data[0]

    # Allow copy if: the set is public, OR the requester is the owner
    if not source["is_public"] and source["user_id"] != x_user_id:
        raise HTTPException(
            status_code=403,
            detail="This card set is private and you are not the owner"
        )

    # Step 2: Verify the target folder exists and belongs to the
requesting user
    folder_check = supabase.table("cardfolders").select("*").eq("id",
target_folder_id).execute()
    if not folder_check.data:
        raise HTTPException(status_code=404, detail="Target folder not
found")
    if folder_check.data[0]["user_id"] != x_user_id:
        raise HTTPException(status_code=403, detail="You do not own the
target folder")

    # Step 3: Create the new card set (owned by requesting user, private
by default)
    new_cardset_data = {
        "cardfolder_id": target_folder_id,
        "user_id": x_user_id,
```

```
        "name": f"Copy of {source['name']}",
        "description": source.get("description"),
        "is_public": False
    }

    new_cardset_result =
supabase.table("cardsets").insert(new_cardset_data).execute()
    if not new_cardset_result.data:
        raise HTTPException(status_code=400, detail="Failed to create card
set copy")

    new_cardset = new_cardset_result.data[0]

    # Step 4: Fetch all cards from the source card set
    source_cards = supabase.table("cards").select("*").eq("cardset_id",
cardset_id).execute()

    # Step 5: Duplicate each card into the new card set
    if source_cards.data:
        new_cards = []
        for card in source_cards.data:
            new_card = {
                "cardset_id": new_cardset["id"],
                "front_text_1": card.get("front_text_1"),
                "front_audio_text_1": card.get("front_audio_text_1"),
                "front_audio_1": card.get("front_audio_1"),
                "front_image": card.get("front_image"),
                "back_text_1": card.get("back_text_1"),
                "back_audio_text_1": card.get("back_audio_text_1"),
                "back_audio_1": card.get("back_audio_1"),
                "back_image": card.get("back_image"),
            }
            new_cards.append(new_card)

        # Batch insert all cards at once for efficiency
        supabase.table("cards").insert(new_cards).execute()

    return new_cardset
```

Notice: The copy always sets `ispublic: False` *on the new card set regardless of the original's visibility — the user must explicitly choose to make their copy public. Also observe the batch insert pattern:* `supabase.table("cards").insert(new` `cards)` sends all duplicated cards in a single database call rather than inserting them one by one in a loop, which is significantly more efficient for large card sets.

IX. FINAL SUMMARY

A. Project Deliverables

❐ 1. Files Created

FILE	PURPOSE
`.env`	Supabase credentials
`.gitignore`	Protects credentials, venv, and cache from version control
`database.py`	Supabase connection — single shared client
`main.py`	Entry point — creates app, includes 4 routers, health check
`models/user_models.py`	Pydantic models for users
`models/cardfolder_models.py`	Pydantic models for folders
`models/cardset_models.py`	Pydantic models for card sets
`models/card_models.py`	Pydantic models for cards
`routers/user_router.py`	User CRUD endpoints
`routers/cardfolder_router.py`	Folder CRUD + ownership endpoints
`routers/cardset_router.py`	Card set CRUD + visibility + copy endpoints
`routers/card_router.py`	Card CRUD + audit trail endpoints
`utils/audio_text.py`	Regex-based audio text generator

- **Total**: 13 files + 3 `init.py` files
- **Total endpoints**: 22 + 1 root health check

B. Multi-User Features Implemented

❐ 1. Complete Feature List

- ✔ User registration and lookup
- ✔ Folder ownership (create, read mine, update owner-only, delete owner-only with cascade)
- ✔ Card set ownership (same pattern)
- ✔ Ownership enforcement on **every** write operation across all entities
- ✔ Public/private visibility via `is_public` flag
- ✔ Visibility filtering on all read operations (public = anyone, private = owner only)
- ✔ Card-level audit trail (`updatedat` + `updated` `by`)
- ✔ Deep copy of public card sets with batch card duplication
- ✔ Simplified authentication via `x-user-id` header

C. How to Run the API

❐ 1. Three Commands

```
cd flashcard-api
workon vintel
uvicorn main:app --reload
```

- Open `localhost:8000/docs` to see and test every endpoint interactively via Swagger UI

D. Four Things to Remember Above All Else

❐ 1. The Data Hierarchy and Where Control Lives

- **➜ Principle**: The hierarchy is `users → cardfolders → cardsets → cards`. Ownership flows down from users. Visibility is controlled at the **card set** level and cascades to all cards within.

❐ 2. CASCADE vs. SET NULL

- **➜ Principle**: `ON DELETE CASCADE` = deleting a parent deletes its children (when the child cannot exist without the parent). `ON DELETE SET NULL` = clearing a reference without destroying data (when the foreign key is informational only).

❐ 3. The API Controls the Audit Trail, Not the Client

- **➜ Principle**: `updatedat` *and* `updated` `by` are set **automatically** by the API during updates — they never appear in Create or Update request bodies. No one can fake their audit trail.

❐ 4. Deep Copy = Complete Independence

- **➜ Principle**: The copy feature produces a **fully independent clone**. Content is preserved exactly, but IDs, timestamps, and audit fields all start fresh. Edits to the copy never affect the original, and vice versa.